

UNIT-4-Iterative Statements

4.1 Iterative statements :

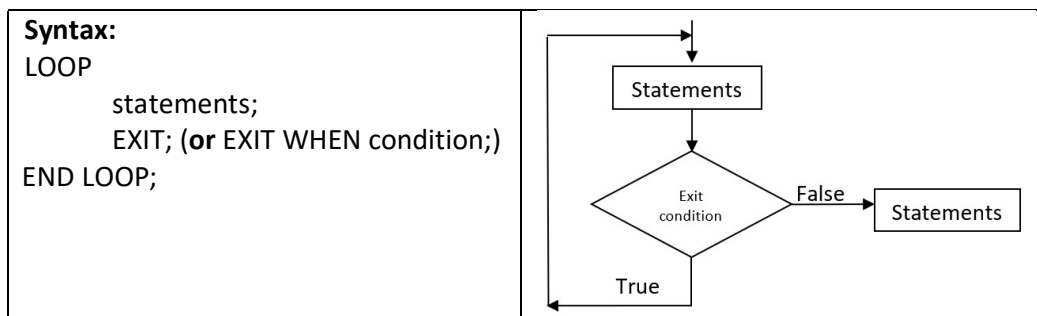
- 4.1.1 Loop..End Loop
- 4.1.2 For.. Loop
- 4.1.3 While Loop
- 4.1.4 EXIT Loop
- 4.1.5 Continue

4.1 Iterative statements

- Iterative statements allow to execute sequential statement(s) multiple times.
- It includes looping statements where same code is repeated till the condition becomes true.
- PL/SQL provides following looping statements:

4.1.1 Loop..End Loop

- The basic loop statement is written between LOOP and END LOOP.
- It repeats the statement(s) multiple times.
- To terminate this loop, EXIT or EXIT WHEN statement is required.



- First of all, numbers of statements are executed and when the condition related to EXIT or EXIT WHEN is executed then the loop is break and move to the statement written after END LOOP.
- There is a need to initialize variable before LOOP and it must be increment or decrement externally within LOOP.

Example: Display first four natural numbers using EXIT

```

declare
    i int:=1;
begin
    loop
        dbms_output.put_line('The value of i is ' || i);
        i:=i+1;
        if i=5 then
            exit;
        end if;
    end loop;
end;

```

```

        end loop;
    end;
/

```

Example: Display first 4 natural numbers using EXIT WHEN

```

declare
    i int:=1;
begin
    loop
        dbms_output.put_line('The value of i is ' || i);
        i:=i+1;
        exit when i=5;
    end loop;
end;
/

```

Output:

```

The value of i is 1
The value of i is 2
The value of i is 3
The value of i is 4

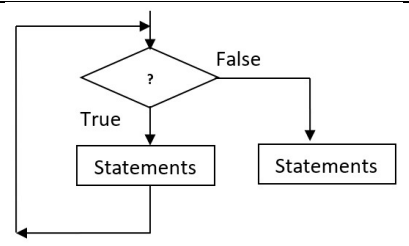
PL/SQL procedure successfully completed.

```

4.1.2 For.. Loop

- For loop is used when you want to repeat the statements fixed numbers of time.

Syntax:
 FOR counter IN [REVERSE] first_val .. last_val
 LOOP
 statements;
 END LOOP;



- PL/SQL initializes “counter=first_val” once and compares it with last_val. If condition becomes true then statements are executed and counter is increment or decrement by **one**.
- To execute from final_val to initial_val (means in descending order) then REVERSE keyword is used.
- Nested for loop is also allowed.

Example: Display the sum of first 5 natural numbers

```

declare
    -- i int; No need to initialize
    s int:=0;
begin
    for i in 1 .. 5 loop
        s:=s+i;
    end loop;
    dbms_output.put_line('Sum is ' || s);
end;
/

```

Output:
Sum is 15

Example: Display first 5 natural numbers in descending order.

```

declare
    i int;
begin
    for i in reverse 1 .. 5 loop
        dbms_output.put_line('Value of i is ' || i);
    end loop;
end;
/

```

Output:

```

Value of i is 5
Value of i is 4
Value of i is 3
Value of i is 2
Value of i is 1
PL/SQL procedure successfully completed.

```

Example: Display the students' rollno, name and marks whose marks is less than 40

```

begin
    dbms_output.put_line('Rollno Name Marks');
    for i in (select * from stud_info where marks<40)
    loop
        dbms_output.put_line(i.rno || ' ' || i.name || ' ' || i.marks);
    end loop;
end;
/

```

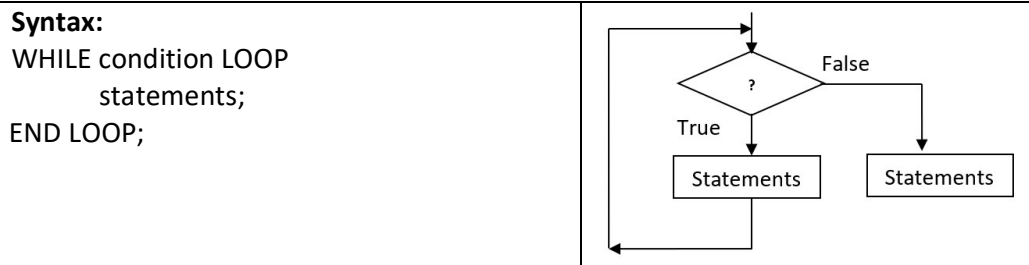
Output:

```
Rollno Name Marks
1 Mayank 35
2 Maya 25
4 Raghav 10

PL/SQL procedure successfully completed.
```

4.1.3 While Loop

- While loop is used when you want to repeat the statements as per specific condition.



- Condition is checked and the numbers of statements are repeated till the condition becomes false.
- There is a need to initialize variable before LOOP and it must be increment or decrement externally within LOOP.

Example: Display the numbers between 10 to 15

```
declare
    i int:=10;
begin
    while i<=15 loop
        dbms_output.put_line('Value of i is ' || i);
        i:=i+1;
    end loop;
end;
/
```

Output:

```
Value of i is 10
Value of i is 11
Value of i is 12
Value of i is 13
Value of i is 14
Value of i is 15

PL/SQL procedure successfully completed.
```

4.2 Control statements

- Control statements are used within looping statements.
- It changes the execution from its normal sequence.

4.2.1 EXIT Loop

- It is used to break the loop and exit from loop.
- It moves control at the next statement after END LOOP.
- Syntax:
 1. EXIT;
 2. EXIT WHEN condition;
- EXIT statement:
 - It is used when you want to exit from loop without specifying any condition.
 - To specify the condition, IF statement is used.
- EXIT WHEN condition;
 - It is used when you want to exit from loop as per specific condition.
 - When the specified condition becomes true then the control transfer to the next statement after END LOOP.

4.2.2 Continue (New feature added in Oracle 11g-Can't work in older versions)

- It is used when you want to return back to the starting of next iteration of loop.
- It ends the current iteration and passes the control to the first line of loop.
- Normally it is written within IF statement.

Example: Display even numbers between 1 to 10.

```
declare
    i int;
begin
    for i in 1..10 loop
        if mod(i,2)<>0 then
            CONTINUE;
        end if;
        dbms_output.put_line('Value of i is ' || i);
    end loop;
end;
/
```

4.3 Goto statement: (Not in syllabus)

- Goto is a jumping statement which allows pointer to move from one location to another within PL/SQL block.
- Syntax:

```
GOTO label;
....
<<label>>
Statements;
Example: Display even numbers between 1 to 10
declare
    i int;
begin
    for i in 1..10 loop
        if mod(i,2)<>0 then
            goto a;
        end if;
        dbms_output.put_line('Value of i is ' || i);
        <<a>>
        NULL;
    end loop;
end;
/
```

Output:

```
Value of i is 2
Value of i is 4
Value of i is 6
Value of i is 8
Value of i is 10
PL/SQL procedure successfully completed.
```